Modularização, CALL, Máscaras Numéricas e Encerramento de Programas

Objetivos da Aula

- Compreender o conceito de modularização em COBOL
- Criar programas compostos por múltiplos módulos
- Utilizar `CALL` estático e dinâmico
- Passar parâmetros entre programas usando `USING`
- Tratar exceções em chamadas com `ON EXCEPTION`
- Utilizar máscaras numéricas para formatação de valores
- Conhecer os comandos de encerramento (`STOP RUN`, `GOBACK`, `EXIT PROGRAM`)
- Desenvolver um pequeno sistema modular

Revisão da Aula Anterior (10 min)

O que já sabemos fazer:

Comando	Função	Exemplo
INSPECT	Analisar/modificar textos	INSPECT TEXTO REPLACING ALL "A" BY "X"
STRING	Concatenar textos	STRING NOME " " SOBRENOME INTO COMPLETO
UNSTRING	Separar textos	UNSTRING DADOS DELIMITED BY ";" INTO CAMPO1
INITIALIZE	Limpar estruturas	INITIALIZE WS-CLIENTE
MOVE CORRESPONDING	Copiar campos com mesmo nome	MOVE CORR WS-ORIGEM TO WS-DESTINO

Problema atual:

```
*>> Um programa com 5000 linhas!
IDENTIFICATION DIVISION.
PROGRAM-ID. SISTEMA-COMPLETO.
...
PROCEDURE DIVISION.
    *>> Cadastro (200 linhas)
    *>> Consulta (200 linhas)
    *>> Cálculo (200 linhas)
    *>> Relatório (200 linhas)
    *>> Menu (200 linhas)
    *>> ... (mais 4000 linhas!)
```

> 🤔 ***Desafio de hoje:*** *Como organizar um grande sistema em módulos menores e reutilizáveis?*

1 – Modularização em COBOL (15 min)

O que é modularização?
É a técnica de **dividir um sistema em programas menores**, cada um com uma responsabilidade específica.

Analogia: Cozinha de Restaurante

```
Restaurante Inteiro (um programa gigante)
├── Cozinha (módulo de preparo)
├── Salão (módulo de atendimento)
├── Caixa (módulo financeiro)
└── Estoque (módulo de controle)
```

Cada área tem sua função específica e pode ser gerenciada separadamente.

Vantagens da Modularização:

Benefício	Descrição
Reutilização	Um módulo pode ser usado em vários programas
Manutenção	Corrigir um erro em apenas um lugar
Organização	Código mais limpo e compreensível
Equipe	Várias pessoas podem trabalhar em paralelo
Testes	Cada módulo pode ser testado isoladamente

Exemplo de Sistema Modular:

Em vez de um único programa gigante:

```
SISTEMA-FOLHA (5000 linhas)
```

Podemos ter:

```
MENU (200 linhas)
CALCULO-SALARIO (500 linhas)
CALCULO-IMPOSTO (300 linhas)
RELATORIO (400 linhas)
CADASTRO-FUNCIONARIO (600 linhas)
```

Total: 2000 linhas distribuídas em 5 módulos!

Como funciona no COBOL?

```
*>> Programa PRINCIPAL
PROCEDURE DIVISION.
    PERFORM MENU.
    CALL "CALCULA-SALARIO" USING WS-FUNCIONARIO.
    CALL "RELATORIO".
    STOP RUN.
```

```
PRINCIPAL
    ↓
    PERFORM MENU
    ↓
    CALL "CALCULA-SALARIO"  → Executa módulo
    ↓
    CALL "RELATORIO"        → Executa módulo
    ↓
    STOP RUN
```

2 – Comandos de Encerramento (20 min)

Os diferentes tipos de "fim"

Em COBOL temos vários comandos para encerrar a execução, cada um com uma finalidade específica.

1. STOP RUN
O que faz: Finaliza **completamente** a execução do programa.

Analogia: Desligar o computador

```
STOP RUN = Desliga tudo, volta para o sistema operacional
```

Sintaxe:

```
STOP RUN.
```

Quando usar:
- No programa principal
- Quando não há retorno necessário

Exemplo:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. PRINCIPAL.

PROCEDURE DIVISION.
    DISPLAY "INICIANDO SISTEMA".
    PERFORM PROCESSAR.
    DISPLAY "FINALIZANDO".
    STOP RUN.    ← Termina tudo
```

2. GOBACK
O que faz: Retorna ao **programa chamador**.

Analogia: Voltar de uma função

```
PROGRAMA A → CALL B → GOBACK → Volta para A
```

Sintaxe:

```
GOBACK.
```

Quando usar:
- Em subprogramas (módulos chamados por CALL)
- Para retornar ao programa principal

Exemplo - Subprograma:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. CALCULA.

PROCEDURE DIVISION USING WS-VALOR.
    COMPUTE WS-VALOR = WS-VALOR * 2.
    GOBACK.    ← Retorna para quem chamou
```

3. EXIT PROGRAM
O que faz: Sai apenas do programa atual, sem retornar valor.

Analogia: Fechar uma janela

```
Sai do programa, mas não retorna dados
```

Sintaxe:

```
EXIT PROGRAM.
```

Quando usar:
- Em programas que não precisam retornar dados
- Para encerrar um programa chamado por CALL

Exemplo:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. VALIDA.

PROCEDURE DIVISION.
    IF WS-OK = "N"
        DISPLAY "DADO INVÁLIDO"
        EXIT PROGRAM    ← Sai sem retornar
    END-IF.
    ...
    GOBACK.    ← Retorna se tudo ok
```

4. CANCEL
O que faz: Remove um programa carregado da memória.

Analogia: Descarregar um aplicativo da memória RAM

```
Libera espaço para outros programas
```

Sintaxe:

```
CANCEL "NOME-PROGRAMA".
```

Quando usar:
- Quando um módulo não será mais usado
- Para liberar memória

Exemplo:

```
CALL "CALCULO".
...
CANCEL "CALCULO".  ← Libera da memória
```

5. TERMINATE
O que faz: Finaliza um relatório em processamento (SORT/MERGE).

Analogia: Fechar o arquivo de relatório

```
Finaliza a geração do relatório
```

Sintaxe:

```
TERMINATE RELATORIO.
```

*> ⚠ **Uso específico:** Principalmente para relatórios com SORT/MERGE. Não usado em programas simples.*

Resumo dos Comandos de Encerramento:

Comando	O que faz	Onde usar
`STOP RUN`	Finaliza tudo	Programa principal
`GOBACK`	Retorna ao chamador	Subprogramas
`EXIT PROGRAM`	Sai do programa	Subprogramas sem retorno
`CANCEL`	Remove da memória	Após usar um módulo
`TERMINATE`	Finaliza relatório	Processamento de relatórios

Exercício de Fixação

Identifique qual comando usar em cada situação:

1. **Programa principal** que não será chamado por ninguém → `STOP RUN`
2. **Subprograma** que calcula um valor e retorna → `GOBACK`
3. **Subprograma** que valida dados e pode sair antes → `EXIT PROGRAM`
4. **Módulo** que não será mais usado → `CANCEL`

3 – CALL Estático (20 min)

O que é CALL?

Comando que **executa outro programa COBOL** a partir do programa atual.

Analogia: Telefonar para alguém

```
Você (programa principal) → Liga (CALL) → Amigo (subprograma)
Amigo atende (executa) → Desliga (GOBACK) → Você continua
```

Estrutura do CALL Estático

Programa Principal (CHAMADOR):

```
IDENTIFICATION DIVISION.
PROGRAM-ID. PRINCIPAL.

PROCEDURE DIVISION.
    DISPLAY "INICIANDO PRINCIPAL".

    CALL "SAUDACAO".    ← Chama o subprograma

    DISPLAY "VOLTEI AO PRINCIPAL".
    STOP RUN.
```

Subprograma (CHAMADO):

```
IDENTIFICATION DIVISION.
PROGRAM-ID. SAUDACAO.

PROCEDURE DIVISION.
    DISPLAY "OLÁ, ESTOU NO SUBPROGRAMA!".
    GOBACK.    ← Retorna ao principal
```

Execução:

```
INICIANDO PRINCIPAL
OLÁ, ESTOU NO SUBPROGRAMA!
VOLTEI AO PRINCIPAL
```

Exemplo Completo - Sistema de Mensagens

Programa Principal:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. SISTEMA-MENSAGENS.
PROCEDURE DIVISION.
    DISPLAY "=== SISTEMA DE MENSAGENS ===".

    CALL "MENSAGEM-BOAS-VINDAS".
    CALL "MENSAGEM-STATUS".
    CALL "MENSAGEM-DESPEDIDA".

    DISPLAY "=== FIM DO SISTEMA ===".
    STOP RUN.
```

Subprograma 1:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MENSAGEM-BOAS-VINDAS.
PROCEDURE DIVISION.
    DISPLAY "BEM-VINDO AO SISTEMA!".
    GOBACK.
```

Subprograma 2:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MENSAGEM-STATUS.
PROCEDURE DIVISION.
    DISPLAY "SISTEMA OPERANDO NORMALMENTE".
    GOBACK.
```

Subprograma 3:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MENSAGEM-DESPEDIDA.
PROCEDURE DIVISION.
    DISPLAY "ATÉ LOGO! VOLTE SEMPRE!".
    GOBACK.
```

Saída:

```
=== SISTEMA DE MENSAGENS ===
BEM-VINDO AO SISTEMA!
SISTEMA OPERANDO NORMALMENTE
ATÉ LOGO! VOLTE SEMPRE!
=== FIM DO SISTEMA ===
```

Fluxo de Execução:

```
PRINCIPAL
    ↓
DISPLAY "=== SISTEMA..."
    ↓
CALL "MENSAGEM-BOAS-VINDAS"
    ↓
    MENSAGEM-BOAS-VINDAS
        DISPLAY "BEM-VINDO..."
        GOBACK
    ↓
CALL "MENSAGEM-STATUS"
    ↓
    MENSAGEM-STATUS
        DISPLAY "SISTEMA OPERANDO..."
        GOBACK
    ↓
CALL "MENSAGEM-DESPEDIDA"
    ↓
    MENSAGEM-DESPEDIDA
        DISPLAY "ATÉ LOGO..."
        GOBACK
    ↓
DISPLAY "=== FIM ==="
    ↓
STOP RUN
```

4 – CALL com Parâmetros (25 min)

Problema: Como enviar dados?
Sem parâmetros, os programas são isolados. Precisamos de uma forma de **passar informações** entre eles.

Analogia: Entregar um pacote

```
Você (principal) → Entrega (USING) → Amigo (subprograma)
Amigo recebe o pacote → Processa → Devolve resultado
```

Sintaxe com Parâmetros

Programa Principal (CHAMADOR):

```
CALL "NOME-SUBPROGRAMA"
    USING parametro1
          parametro2
          parametro3.
```

Subprograma (CHAMADO):

```
IDENTIFICATION DIVISION.
PROGRAM-ID. NOME-SUBPROGRAMA.

DATA DIVISION.
WORKING-STORAGE SECTION.
*>> Variáveis internas

LINKAGE SECTION.    ← Seção especial para parâmetros
01 LK-PARAM1 PIC 9(5).
01 LK-PARAM2 PIC X(10).
01 LK-PARAM3 PIC 9(3).

PROCEDURE DIVISION USING LK-PARAM1
                         LK-PARAM2
                         LK-PARAM3.
*>> Processa usando os parâmetros
    GOBACK.
```

Exemplo 1 - Passando um Parâmetro

Programa Principal:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. PRINCIPAL.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME PIC X(30) VALUE "JOAO SILVA".

PROCEDURE DIVISION.
    DISPLAY "PRINCIPAL - NOME: " WS-NOME.

    CALL "EXIBE-NOME"
        USING WS-NOME.

    DISPLAY "PRINCIPAL - VOLTEI".
    STOP RUN.
```

Subprograma:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXIBE-NOME.

DATA DIVISION.
LINKAGE SECTION.
01 LK-NOME PIC X(30).

PROCEDURE DIVISION USING LK-NOME.
    DISPLAY "SUBPROGRAMA - RECEBI: " LK-NOME.
    GOBACK.
```

Execução:

```
PRINCIPAL - NOME: JOAO SILVA
SUBPROGRAMA - RECEBI: JOAO SILVA
PRINCIPAL - VOLTEI
```

Exemplo 2 - Passando Múltiplos Parâmetros

Programa Principal:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. CALCULO-PRINCIPAL.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NUM1 PIC 9(3) VALUE 10.
01 WS-NUM2 PIC 9(3) VALUE 20.
01 WS-SOMA PIC 9(5).

PROCEDURE DIVISION.
    DISPLAY "PRINCIPAL - " WS-NUM1 " + " WS-NUM2.

    CALL "CALCULA-SOMA"
        USING WS-NUM1
              WS-NUM2
              WS-SOMA.

    DISPLAY "PRINCIPAL - RESULTADO: " WS-SOMA.
    STOP RUN.
```

Subprograma:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. CALCULA-SOMA.

DATA DIVISION.
LINKAGE SECTION.
01 LK-NUM1 PIC 9(3).
01 LK-NUM2 PIC 9(3).
01 LK-SOMA PIC 9(5).

PROCEDURE DIVISION USING LK-NUM1
                         LK-NUM2
                         LK-SOMA.
    DISPLAY "SUBPROGRAMA - SOMANDO...".

    ADD LK-NUM1 TO LK-NUM2
        GIVING LK-SOMA.

    GOBACK.
```

Execução:

```
PRINCIPAL - 10 + 20
SUBPROGRAMA - SOMANDO...
PRINCIPAL - RESULTADO: 30
```

Exemplo 3 - Processando e Retornando

Programa Principal:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. ANALISE-CLIENTE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-CLIENTE.
   05 WS-NOME      PIC X(30).
   05 WS-IDADE     PIC 9(3).
   05 WS-SITUACAO  PIC X(20).

PROCEDURE DIVISION.
    DISPLAY "DIGITE O NOME:".
    ACCEPT WS-NOME.
    DISPLAY "DIGITE A IDADE:".
    ACCEPT WS-IDADE.

    CALL "CLASSIFICA-IDADE"
```

```
        USING WS-IDADE
              WS-SITUACAO.

    DISPLAY "CLIENTE: " WS-NOME.
    DISPLAY "SITUAÇÃO: " WS-SITUACAO.

    STOP RUN.
```

Subprograma:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. CLASSIFICA-IDADE.

DATA DIVISION.
LINKAGE SECTION.
01 LK-IDADE     PIC 9(3).
01 LK-SITUACAO  PIC X(20).

PROCEDURE DIVISION USING LK-IDADE
                         LK-SITUACAO.
    IF LK-IDADE < 18
        MOVE "MENOR DE IDADE" TO LK-SITUACAO
    ELSE
        IF LK-IDADE < 65
            MOVE "ADULTO" TO LK-SITUACAO
        ELSE
            MOVE "IDOSO" TO LK-SITUACAO
        END-IF
    END-IF.

    GOBACK.
```

Execução:

```
DIGITE O NOME:
MARIA SANTOS
DIGITE A IDADE:
35
CLIENTE: MARIA SANTOS
SITUAÇÃO: ADULTO
```

Regras Importantes do CALL com USING:

Regra	Explicação
Ordem importa	A ordem dos parâmetros deve ser a mesma
Tipos compatíveis	Os tipos devem ser compatíveis
Tamanhos	Tamanhos devem ser iguais ou compatíveis
BY REFERENCE	Padrão: alterações afetam a variável original
BY CONTENT	Cópia: alterações não afetam a original

BY CONTENT (proteger dados):

```
CALL "PROCESSAR"
    USING BY CONTENT WS-VALOR
    ...
```

5 – CALL com Múltiplos Parâmetros (10 min)

Exemplo - Calculadora Modular

Programa Principal (Menu):

```
IDENTIFICATION DIVISION.
PROGRAM-ID. CALCULADORA-MODULAR.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NUM1     PIC 9(5)V99.
01 WS-NUM2     PIC 9(5)V99.
01 WS-RESULT   PIC 9(6)V99.
01 WS-OPCAO    PIC 9.

PROCEDURE DIVISION.
    PERFORM UNTIL WS-OPCAO = 5
        DISPLAY "=== CALCULADORA MODULAR ==="
        DISPLAY "1 - SOMAR"
        DISPLAY "2 - SUBTRAIR"
        DISPLAY "3 - MULTIPLICAR"
        DISPLAY "4 - DIVIDIR"
        DISPLAY "5 - SAIR"
        DISPLAY "OPÇÃO:"
```

```
        ACCEPT WS-OPCAO

        IF WS-OPCAO NOT = 5
            DISPLAY "PRIMEIRO NÚMERO:"
            ACCEPT WS-NUM1
            DISPLAY "SEGUNDO NÚMERO:"
            ACCEPT WS-NUM2

            EVALUATE WS-OPCAO
                WHEN 1
                    CALL "MOD-SOMA"
                        USING WS-NUM1 WS-NUM2 WS-RESULT
                WHEN 2
                    CALL "MOD-SUBTRAI"
                        USING WS-NUM1 WS-NUM2 WS-RESULT
                WHEN 3
                    CALL "MOD-MULTIPLICA"
                        USING WS-NUM1 WS-NUM2 WS-RESULT
                WHEN 4
                    CALL "MOD-DIVIDE"
                        USING WS-NUM1 WS-NUM2 WS-RESULT
                WHEN OTHER
                    DISPLAY "OPÇÃO INVÁLIDA!"
            END-EVALUATE

            DISPLAY "RESULTADO: " WS-RESULT
            DISPLAY " "
        END-IF
    END-PERFORM.

    DISPLAY "CALCULADORA ENCERRADA".
    STOP RUN.
```

Módulo de Soma:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MOD-SOMA.

DATA DIVISION.
LINKAGE SECTION.
01 LK-NUM1   PIC 9(5)V99.
01 LK-NUM2   PIC 9(5)V99.
01 LK-RESULT PIC 9(6)V99.

PROCEDURE DIVISION USING LK-NUM1
                         LK-NUM2
```

```
                          LK-RESULT.
    ADD LK-NUM1 TO LK-NUM2
        GIVING LK-RESULT.
    GOBACK.
```

Módulo de Subtração:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MOD-SUBTRAI.

DATA DIVISION.
LINKAGE SECTION.
01 LK-NUM1   PIC 9(5)V99.
01 LK-NUM2   PIC 9(5)V99.
01 LK-RESULT PIC 9(6)V99.

PROCEDURE DIVISION USING LK-NUM1
                         LK-NUM2
                         LK-RESULT.
    SUBTRACT LK-NUM2 FROM LK-NUM1
        GIVING LK-RESULT.
    GOBACK.
```

Módulo de Multiplicação:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MOD-MULTIPLICA.

DATA DIVISION.
LINKAGE SECTION.
01 LK-NUM1   PIC 9(5)V99.
01 LK-NUM2   PIC 9(5)V99.
01 LK-RESULT PIC 9(6)V99.

PROCEDURE DIVISION USING LK-NUM1
                         LK-NUM2
                         LK-RESULT.
    MULTIPLY LK-NUM1 BY LK-NUM2
        GIVING LK-RESULT.
    GOBACK.
```

Módulo de Divisão:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MOD-DIVIDE.

DATA DIVISION.
```

```
LINKAGE SECTION.
01 LK-NUM1   PIC 9(5)V99.
01 LK-NUM2   PIC 9(5)V99.
01 LK-RESULT PIC 9(6)V99.

PROCEDURE DIVISION USING LK-NUM1
                         LK-NUM2
                         LK-RESULT.
    IF LK-NUM2 = 0
        MOVE 0 TO LK-RESULT
        DISPLAY "ERRO! DIVISÃO POR ZERO!"
    ELSE
        DIVIDE LK-NUM1 BY LK-NUM2
            GIVING LK-RESULT ROUNDED
    END-IF.
    GOBACK.
```

6 – CALL Dinâmico (10 min)

O que é CALL Dinâmico?

O nome do programa a ser chamado é **armazenado em uma variável**, permitindo escolha em tempo de execução.

Analogia: Agenda de Contatos

```
Você escolhe: "Ligar para Amigo" → Nome é "AMIGO"
Amigo atende (executa)
Depois: "Ligar para Família" → Nome é "FAMILIA"
```

Sintaxe:

```
01 WS-NOME-MODULO PIC X(20).

MOVE "MOD-SOMA" TO WS-NOME-MODULO.
CALL WS-NOME-MODULO USING WS-NUM1 WS-NUM2 WS-RESULT.
```

Exemplo - Menu Dinâmico:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MENU-DINAMICO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-OPCAO    PIC 9.
01 WS-MODULO   PIC X(20).
01 WS-NUM1     PIC 9(5)V99.
```

```
01 WS-NUM2     PIC 9(5)V99.
01 WS-RESULT   PIC 9(6)V99.

PROCEDURE DIVISION.
    DISPLAY "=== MENU DINÂMICO ===".
    DISPLAY "1 - SOMAR".
    DISPLAY "2 - SUBTRAIR".
    DISPLAY "OPÇÃO:".
    ACCEPT WS-OPCAO.

    DISPLAY "NÚMERO 1:".
    ACCEPT WS-NUM1.
    DISPLAY "NÚMERO 2:".
    ACCEPT WS-NUM2.

    *>> Escolhe o módulo dinamicamente
    IF WS-OPCAO = 1
        MOVE "MOD-SOMA" TO WS-MODULO
    ELSE
        MOVE "MOD-SUBTRAI" TO WS-MODULO
    END-IF.

    *>> Chama o módulo escolhido
    CALL WS-MODULO
        USING WS-NUM1 WS-NUM2 WS-RESULT.

    DISPLAY "RESULTADO: " WS-RESULT.
    STOP RUN.
```

Vantagem do CALL Dinâmico:

Estático	Dinâmico
Nome fixo no código	Nome em variável
Não muda em execução	Pode mudar em execução
Mais rápido	Mais flexível

> 💡 ***Dica**: Use dinâmico quando precisar de flexibilidade!*

7 – Tratamento de Exceções em CALL (10 min)

O que é ON EXCEPTION?

Trata erros quando o programa chamado **não existe** ou **não pode ser carregado**.

Analogia: Número de telefone errado

```
Você liga → Número não existe → Operadora: "Número inválido"
```

Sintaxe:

```
CALL "NOME-MODULO"
    ON EXCEPTION
        comando(s) para ERRO
    NOT ON EXCEPTION
        comando(s) para SUCESSO
END-CALL.
```

Exemplo - Chamada Segura:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. CHAMADA-SEGURA.

PROCEDURE DIVISION.
    DISPLAY "TENTANDO CHAMAR MÓDULO...".

    CALL "MODULO-QUE-NAO-EXISTE"
        ON EXCEPTION
            DISPLAY "ERRO! MÓDULO NÃO ENCONTRADO!"
            DISPLAY "VERIFIQUE O NOME DO PROGRAMA."
        NOT ON EXCEPTION
            DISPLAY "MÓDULO EXECUTADO COM SUCESSO!"
    END-CALL.

    DISPLAY "PROGRAMA CONTINUA...".
    STOP RUN.
```

Saída:

```
TENTANDO CHAMAR MÓDULO...
ERRO! MÓDULO NÃO ENCONTRADO!
VERIFIQUE O NOME DO PROGRAMA.
PROGRAMA CONTINUA...
```

Exemplo - Com Menu e Tratamento:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MENU-SEGURO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-OPCAO    PIC 9.
01 WS-MODULO   PIC X(20).

PROCEDURE DIVISION.
    DISPLAY "1 - CALCULAR SALÁRIO".
    DISPLAY "2 - CALCULAR IMPOSTO".
    DISPLAY "3 - RELATÓRIO".
    DISPLAY "OPÇÃO:".
    ACCEPT WS-OPCAO.

    EVALUATE WS-OPCAO
        WHEN 1
            MOVE "CALC-SALARIO" TO WS-MODULO
        WHEN 2
            MOVE "CALC-IMPOSTO" TO WS-MODULO
        WHEN 3
            MOVE "GERA-RELATORIO" TO WS-MODULO
        WHEN OTHER
            DISPLAY "OPÇÃO INVÁLIDA!"
            STOP RUN
    END-EVALUATE.

    CALL WS-MODULO
        ON EXCEPTION
            DISPLAY "ERRO! MÓDULO '" WS-MODULO "' NÃO ENCONTRADO!"
            DISPLAY "CONTATE O SUPORTE TÉCNICO."
        NOT ON EXCEPTION
            DISPLAY "MÓDULO EXECUTADO COM SUCESSO!"
    END-CALL.

    STOP RUN.
```

8 – Máscaras Numéricas (20 min)

O problema dos números "crus"

Sem formatação, números são difíceis de ler:

```
1234567.89  ← Não é amigável!
```

Com máscara:

```
1.234.567,89  ← Muito melhor!
R$ 1.234,56  ← Formato monetário
```

O que são máscaras numéricas?

São **PICTUREs especiais** que formatam números para exibição.

Analogia: Moldura para quadro

```
Número bruto → Moldura (máscara) → Número formatado
```

Tipos de Máscaras:

Símbolo	O que faz
`Z`	Substitui zero por espaço
`9`	Força exibição do dígito
`.`	Ponto decimal (separador)
`,`	Separador de milhares
`$`	Símbolo monetário
`-`	Sinal negativo/positivo
`+`	Sinal sempre exibido
`B`	Espaço em branco
`0`	Força zero à esquerda

Exemplo 1 - Máscara Básica

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MASCARA-BASICA.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-VALOR     PIC 9(6)V99 VALUE 1234567.
01 WS-EDIT-1    PIC ZZ.ZZZ.ZZ9,99.
01 WS-EDIT-2    PIC $ZZZ.ZZ9,99.
01 WS-EDIT-3    PIC ZZZ.ZZ9,99-.

PROCEDURE DIVISION.
    MOVE WS-VALOR TO WS-EDIT-1.
    DISPLAY "VALOR: " WS-EDIT-1.

    MOVE WS-VALOR TO WS-EDIT-2.
    DISPLAY "MONETÁRIO: " WS-EDIT-2.

    *>> Valor negativo
    MOVE -1234567 TO WS-VALOR.
    MOVE WS-VALOR TO WS-EDIT-3.
    DISPLAY "NEGATIVO: " WS-EDIT-3.

    STOP RUN.
```

Saída:

```
VALOR: 12.345,67
MONETÁRIO: $12.345,67
NEGATIVO: 12.345,67-
```

Exemplo 2 - Máscara com Sinal

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MASCARA-SINAL.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-POSITIVO   PIC S9(5)V99 VALUE 123456.
01 WS-NEGATIVO   PIC S9(5)V99 VALUE -123456.
01 WS-EDIT-POS   PIC ZZZ.ZZ9,99+.
01 WS-EDIT-NEG   PIC ZZZ.ZZ9,99-.
01 WS-EDIT-BOTH  PIC ZZZ.ZZ9,99.

PROCEDURE DIVISION.
    MOVE WS-POSITIVO TO WS-EDIT-POS.
    DISPLAY "POSITIVO: " WS-EDIT-POS.
```

```
    MOVE WS-NEGATIVO TO WS-EDIT-NEG.
    DISPLAY "NEGATIVO: " WS-EDIT-NEG.

    MOVE WS-POSITIVO TO WS-EDIT-BOTH.
    DISPLAY "POSITIVO: " WS-EDIT-BOTH.

    MOVE WS-NEGATIVO TO WS-EDIT-BOTH.
    DISPLAY "NEGATIVO: " WS-EDIT-BOTH.

    STOP RUN.
```

Saída:

```
POSITIVO: 123.456+
NEGATIVO: 123.456-
POSITIVO: 123.456
NEGATIVO: -123.456
```

Exemplo 3 - Máscara para Relatórios

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MASCARA-RELATORIO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-DADOS.
   05 WS-NOME      PIC X(30) VALUE "JOAO SILVA".
   05 WS-SALARIO   PIC 9(7)V99 VALUE 12500.50.
   05 WS-COMISSAO  PIC 9(7)V99 VALUE 1500.75.
   05 WS-TOTAL     PIC 9(7)V99.

01 WS-RELATORIO.
   05 WS-NOME-EDIT      PIC X(30).
   05 WS-SALARIO-EDIT   PIC ZZZ.ZZ9,99.
   05 WS-COMISSAO-EDIT  PIC ZZZ.ZZ9,99.
   05 WS-TOTAL-EDIT     PIC $$$,$$$,$$9.99.

PROCEDURE DIVISION.
    ADD WS-SALARIO TO WS-COMISSAO
        GIVING WS-TOTAL.

    MOVE WS-NOME      TO WS-NOME-EDIT.
    MOVE WS-SALARIO   TO WS-SALARIO-EDIT.
    MOVE WS-COMISSAO  TO WS-COMISSAO-EDIT.
    MOVE WS-TOTAL     TO WS-TOTAL-EDIT.
```

```
    DISPLAY "================================".
    DISPLAY "RELATÓRIO DE COMISSÃO".
    DISPLAY "================================".
    DISPLAY "NOME: " WS-NOME-EDIT.
    DISPLAY "SALÁRIO: R$ " WS-SALARIO-EDIT.
    DISPLAY "COMISSÃO: R$ " WS-COMISSAO-EDIT.
    DISPLAY "TOTAL: " WS-TOTAL-EDIT.
    DISPLAY "================================".

    STOP RUN.
```

Saída:

```
================================
RELATÓRIO DE COMISSÃO
================================
NOME: JOAO SILVA
SALÁRIO: R$ 12.500,50
COMISSÃO: R$ 1.500,75
TOTAL: $14,001.25
================================
```

Máscaras Especiais:

Máscara	Descrição	Exemplo de entrada	Saída
`ZZZ.ZZ9,99`	Número com separadores	1234567	12.345,67
`$ZZZ.ZZ9,99`	Monetário	1234567	$12.345,67
`ZZZ.ZZ9,99-`	Negativo com sinal no final	-1234567	12.345,67-
`ZZZ.ZZ9,99+`	Positivo com sinal	1234567	12.345,67+
`$$$,$$$,$$9.99`	Monetário com zeros fixos	12345.67	$12,345.67
`ZZZ.ZZ9,99DB`	Débito/Crédito	-1234567	12.345,67 DB

Exercício Guiado (10 min)

Programa - Formatação de Valores

Crie um programa que:
1. Receba um valor numérico
2. Exiba em 3 formatos diferentes:
   - Número simples (com separadores)
   - Formato monetário (R$)
   - Formato com sinal

Resolução:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXERCICIO-MASCARA.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-VALOR     PIC S9(6)V99.
01 WS-EDIT-1    PIC ZZ.ZZ9,99.
01 WS-EDIT-2    PIC R$ ZZ.ZZ9,99.
01 WS-EDIT-3    PIC ZZ.ZZ9,99-.

PROCEDURE DIVISION.
    DISPLAY "DIGITE UM VALOR:".
    ACCEPT WS-VALOR.

    MOVE WS-VALOR TO WS-EDIT-1.
    MOVE WS-VALOR TO WS-EDIT-2.
    MOVE WS-VALOR TO WS-EDIT-3.

    DISPLAY "==========================".
    DISPLAY "VALOR: " WS-EDIT-1.
    DISPLAY "MOEDA: " WS-EDIT-2.
    IF WS-VALOR < 0
        DISPLAY "SINAL: NEGATIVO"
    ELSE
        DISPLAY "SINAL: POSITIVO"
    END-IF.
    DISPLAY "==========================".

    STOP RUN.
```

Atividade Prática da Aula (35 min)

🎯 **Sistema Modular de Folha de Pagamento**

Objetivo:
Criar um sistema completo com 2 módulos que processa a folha de pagamento.

Estrutura do Sistema:

```
SISTEMA-FOLHA
├── FOLHA-PRINCIPAL (Programa Principal)
│   ├── Menu
│   ├── Entrada de dados
│   └── Chama módulo de cálculo
└── CALCULA-FOLHA (Subprograma)
    ├── Recebe dados do funcionário
    ├── Calcula salário bruto
    ├── Calcula descontos
    ├── Calcula salário líquido
    └── Retorna resultado
```

Programa Principal - FOLHA-PRINCIPAL:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. FOLHA-PRINCIPAL.

       DATA DIVISION.
       WORKING-STORAGE SECTION.
       *>> DADOS DO FUNCIONÁRIO
       01 WS-FUNCIONARIO.
          05 WS-NOME          PIC X(40).
          05 WS-SALARIO-BASE  PIC 9(7)V99.
          05 WS-HORAS-EXTRAS  PIC 9(3).
          05 WS-VALOR-HORA    PIC 9(5)V99.
          05 WS-VENDAS        PIC 9(7)V99.

       *>> RESULTADOS DO CÁLCULO
       01 WS-RESULTADO.
          05 WS-SALARIO-BRUTO    PIC 9(7)V99.
          05 WS-SALARIO-LIQUIDO  PIC 9(7)V99.
          05 WS-INSS             PIC 9(7)V99.
          05 WS-COMISSAO         PIC 9(7)V99.

       *>> MÁSCARAS PARA EXIBIÇÃO
       01 WS-EXIBE.
          05 WS-NOME-EDIT        PIC X(40).
          05 WS-SALARIO-EDIT     PIC ZZZ.ZZ9,99.
```

```
           05 WS-BRUTO-EDIT       PIC ZZZ.ZZ9,99.
           05 WS-LIQUIDO-EDIT     PIC ZZZ.ZZ9,99.
           05 WS-INSS-EDIT        PIC ZZZ.ZZ9,99.
           05 WS-COMISSAO-EDIT    PIC ZZZ.ZZ9,99.

       *>> CONTROLE
       01 WS-OPCAO PIC X(1).
       01 WS-STATUS PIC X(1).

       PROCEDURE DIVISION.
           PERFORM CABECALHO.

           PERFORM UNTIL WS-OPCAO = "N"
               PERFORM ENTRADA-DADOS
               PERFORM CALCULAR-FOLHA
               PERFORM EXIBIR-RESULTADO
               PERFORM PERGUNTAR-CONTINUAR
           END-PERFORM.

           DISPLAY "======================================".
           DISPLAY "        PROGRAMA ENCERRADO".
           DISPLAY "======================================".
           STOP RUN.

       *> ======================================
       *> CABECALHO
       *> ======================================
       CABECALHO.
           DISPLAY "======================================".
           DISPLAY "   SISTEMA DE FOLHA DE PAGAMENTO".
           DISPLAY "   VERSÃO MODULAR".
           DISPLAY "======================================".
           DISPLAY " ".
           EXIT.

       *> ======================================
       *> ENTRADA DE DADOS
       *> ======================================
       ENTRADA-DADOS.
           DISPLAY "DADOS DO FUNCIONÁRIO".
           DISPLAY "-----------------------------------".
           DISPLAY "NOME COMPLETO:".
           ACCEPT WS-NOME.

           DISPLAY "SALÁRIO BASE:".
           ACCEPT WS-SALARIO-BASE.
```

```
           DISPLAY "VALOR DA HORA TRABALHADA:".
           ACCEPT WS-VALOR-HORA.

           DISPLAY "QUANTIDADE DE HORAS EXTRAS:".
           ACCEPT WS-HORAS-EXTRAS.

           DISPLAY "TOTAL DE VENDAS NO MÊS:".
           ACCEPT WS-VENDAS.

           DISPLAY " ".
           EXIT.

       *> ======================================
       *> CHAMADA DO MÓDULO DE CÁLCULO
       *> ======================================
       CALCULAR-FOLHA.
           DISPLAY "PROCESSANDO FOLHA...".

           *>> Chama o módulo de cálculo
           CALL "CALCULA-FOLHA"
               USING WS-FUNCIONARIO
                     WS-RESULTADO
               ON EXCEPTION
                   MOVE "E" TO WS-STATUS
                   DISPLAY "ERRO! MÓDULO DE CÁLCULO NÃO ENCONTRADO!"
               NOT ON EXCEPTION
                   MOVE "S" TO WS-STATUS
                   DISPLAY "CÁLCULO REALIZADO COM SUCESSO!"
           END-CALL.

           DISPLAY " ".
           EXIT.

       *> ======================================
       *> EXIBIR RESULTADO
       *> ======================================
       EXIBIR-RESULTADO.
           IF WS-STATUS = "S"
               *>> Aplicar máscaras
               MOVE WS-NOME           TO WS-NOME-EDIT
               MOVE WS-SALARIO-BASE   TO WS-SALARIO-EDIT
               MOVE WS-SALARIO-BRUTO  TO WS-BRUTO-EDIT
               MOVE WS-SALARIO-LIQUIDO TO WS-LIQUIDO-EDIT
               MOVE WS-INSS           TO WS-INSS-EDIT
               MOVE WS-COMISSAO       TO WS-COMISSAO-EDIT
```

```
            DISPLAY "======================================".
            DISPLAY "        FOLHA DE PAGAMENTO".
            DISPLAY "======================================".
            DISPLAY " ".
            DISPLAY "FUNCIONÁRIO: " WS-NOME-EDIT.
            DISPLAY " ".
            DISPLAY "SALÁRIO BASE : R$ " WS-SALARIO-EDIT.
            DISPLAY "COMISSÃO     : R$ " WS-COMISSAO-EDIT.
            DISPLAY "--------------------------------------".
            DISPLAY "SALÁRIO BRUTO: R$ " WS-BRUTO-EDIT.
            DISPLAY " ".
            DISPLAY "DESCONTOS".
            DISPLAY "INSS         : R$ " WS-INSS-EDIT.
            DISPLAY "--------------------------------------".
            DISPLAY "SALÁRIO LÍQUIDO: R$ " WS-LIQUIDO-EDIT.
            DISPLAY " ".
            DISPLAY "======================================".
        ELSE
            DISPLAY "ERRO AO CALCULAR FOLHA!"
        END-IF.
        EXIT.

    *> ======================================
    *> PERGUNTAR CONTINUAR
    *> ======================================
    PERGUNTAR-CONTINUAR.
        DISPLAY " ".
        DISPLAY "DESEJA CALCULAR OUTRO FUNCIONÁRIO? (S/N)".
        ACCEPT WS-OPCAO.
        DISPLAY " ".
        EXIT.
```

Subprograma - CALCULA-FOLHA:

```
    IDENTIFICATION DIVISION.
    PROGRAM-ID. CALCULA-FOLHA.

    DATA DIVISION.
    WORKING-STORAGE SECTION.
    *>> Constantes
    78 PERCENTUAL-HORA-EXTRA VALUE 1.5.
    78 PERCENTUAL-COMISSAO  VALUE 0.05.
    78 PERCENTUAL-INSS      VALUE 0.11.

    *>> Variáveis internas
```

```
       01 WS-VALOR-EXTRA PIC 9(7)V99.
       01 WS-TOTAL-BRUTO PIC 9(7)V99.

       *>> Parâmetros recebidos
       LINKAGE SECTION.
       01 LK-FUNCIONARIO.
          05 LK-NOME          PIC X(40).
          05 LK-SALARIO-BASE  PIC 9(7)V99.
          05 LK-HORAS-EXTRAS  PIC 9(3).
          05 LK-VALOR-HORA    PIC 9(5)V99.
          05 LK-VENDAS        PIC 9(7)V99.

       01 LK-RESULTADO.
          05 LK-SALARIO-BRUTO    PIC 9(7)V99.
          05 LK-SALARIO-LIQUIDO  PIC 9(7)V99.
          05 LK-INSS             PIC 9(7)V99.
          05 LK-COMISSAO         PIC 9(7)V99.

       PROCEDURE DIVISION USING LK-FUNCIONARIO
                                LK-RESULTADO.
           DISPLAY "   * CALCULANDO FOLHA...".

           *>> 1. Calcular valor das horas extras
           MULTIPLY LK-VALOR-HORA BY PERCENTUAL-HORA-EXTRA
               GIVING WS-VALOR-EXTRA.

           MULTIPLY LK-HORAS-EXTRAS BY WS-VALOR-EXTRA
               GIVING WS-VALOR-EXTRA.

           *>> 2. Calcular salário bruto (base + extras)
           ADD LK-SALARIO-BASE TO WS-VALOR-EXTRA
               GIVING LK-SALARIO-BRUTO.

           *>> 3. Calcular comissão
           MULTIPLY LK-VENDAS BY PERCENTUAL-COMISSAO
               GIVING LK-COMISSAO.

           *>> 4. Calcular total bruto
           ADD LK-SALARIO-BRUTO TO LK-COMISSAO
               GIVING WS-TOTAL-BRUTO.

           *>> 5. Calcular INSS
           MULTIPLY WS-TOTAL-BRUTO BY PERCENTUAL-INSS
               GIVING LK-INSS.

           *>> 6. Calcular salário líquido
```

```
           SUBTRACT LK-INSS FROM WS-TOTAL-BRUTO
               GIVING LK-SALARIO-LIQUIDO.

           DISPLAY "   * CÁLCULO FINALIZADO.".

           GOBACK.
```

Saída Esperada:

```
========================================
   SISTEMA DE FOLHA DE PAGAMENTO
   VERSÃO MODULAR
========================================

DADOS DO FUNCIONÁRIO
----------------------------------------
NOME COMPLETO:
MARIA SANTOS
SALÁRIO BASE:
3000.00
VALOR DA HORA TRABALHADA:
20.00
QUANTIDADE DE HORAS EXTRAS:
10
TOTAL DE VENDAS NO MÊS:
15000.00

PROCESSANDO FOLHA...
   * CALCULANDO FOLHA...
   * CÁLCULO FINALIZADO.
CÁLCULO REALIZADO COM SUCESSO!

========================================
        FOLHA DE PAGAMENTO
========================================

FUNCIONÁRIO: MARIA SANTOS

SALÁRIO BASE : R$  3.000,00
COMISSÃO     : R$    750,00
----------------------------------------
SALÁRIO BRUTO: R$  4.050,00

DESCONTOS
INSS         : R$    445,50
----------------------------------------
```

```
SALÁRIO LÍQUIDO: R$  3.604,50

========================================

DESEJA CALCULAR OUTRO FUNCIONÁRIO? (S/N)
N

========================================
        PROGRAMA ENCERRADO
========================================
```

Desafio Extra (Opcional)

🏆 **Sistema de Módulos com CALL Dinâmico**

Criar um sistema onde o usuário escolhe entre:

1. Cálculo de Salário
2. Cálculo de Juros
3. Cálculo de IMC

O programa deve:
- Usar CALL dinâmico
- Ter tratamento de exceções
- Usar máscaras numéricas
- Ser modular

Encerramento do Curso

O que aprendemos em 6 aulas:

Aula	Conteúdo
Aula 1	Introdução, DISPLAY, ACCEPT
Aula 2	Variáveis, PICTURE, WORKING-STORAGE
Aula 3	Operações matemáticas, MOVE, SET
Aula 4	Decisão (IF, EVALUATE) e Repetição (PERFORM)
Aula 5	Manipulação de strings (STRING, UNSTRING, INSPECT)
Aula 6	Modularização (CALL), Máscaras Numéricas

📈 Você está pronto para:

- ✅ Desenvolver programas COBOL completos
- ✅ Criar sistemas modulares e organizados
- ✅ Manipular dados de negócio
- ✅ Formatar números para relatórios
- ✅ Processar textos e arquivos

🚀 Próximos Passos Sugeridos:

Para aprofundamento, estude:

1. **Arquivos Sequenciais** - Leitura/escrita de arquivos
2. **Arquivos Indexados** - Acesso direto por chave
3. **SORT e MERGE** - Ordenação e junção de dados
4. **Tabelas Avançadas** - OCCURS DEPENDING ON
5. **Busca Binária** - SEARCH e SEARCH ALL
6. **JCL** - Job Control Language
7. **DB2/SQL** - Acesso a banco de dados
8. **Mainframe** - Ambiente z/OS

Parabéns por concluir o curso! 👷 👷

Agora você tem uma base sólida para trabalhar com COBOL em sistemas corporativos. Continue praticando e explorando!